

WEEK-II

REAL WORLD EXAMPLE

faculty.washington.edu/sbrunton/me565/pdf/L29secure.pdf

EXTENDED YALE FACE DATASET



- Pixel is the smallest unit of a digital image.

EACH Data point $x_i \in \mathbb{R}^{\sim 32000}$

$d \approx 32000$; $n \approx 20000$

no. of pixels in each image

PRINCIPAL COMPONENTS

Most
common
template →



PC1



PC2



...

...



PCK

variance decreasing →
details increasing →



TEST IMAGE 1



TEST IMAGE 2

ISSUES / CONCERNS with PCA

- TIME COMPLEXITY - Finding the Eigen vectors and Eigen values.

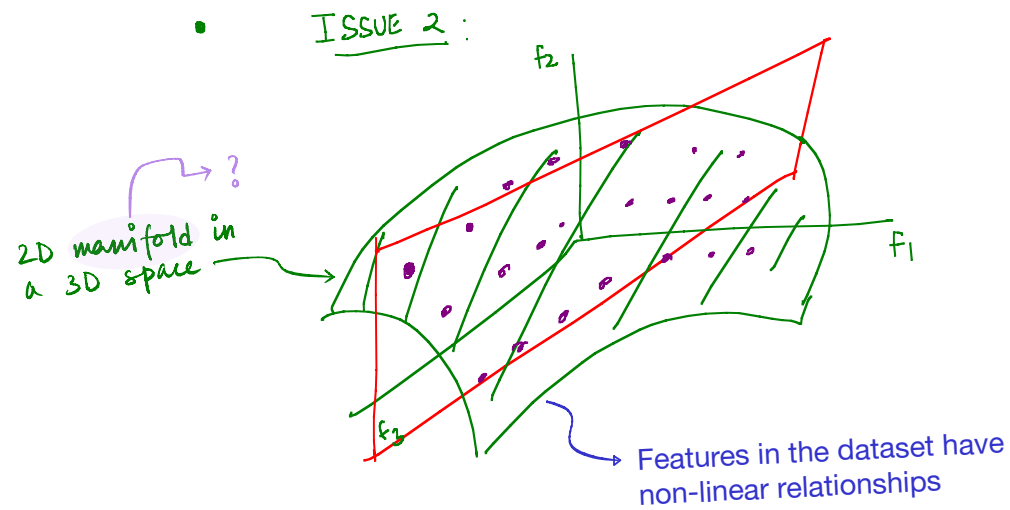
co-variance matrix $\leftarrow C \in \mathbb{R}^{d \times d}$

Typically $O(d^3)$
↓
"Big-Oh(O)"

- Issue when d is large

Example: Face recognition (Eigenfaces)

- **Big-O** notation defines the asymptotic upper bound of an algorithm's running time, which is measured by how the number of basic operations grows relative to the input size.



Data may not necessarily live in a low-dimensional **LINEAR** subspace.

SURPRISING RESULT : Same solution to both issues!

Issue 1:Large d

$$[d \gg n]$$

$\swarrow$ # features $\searrow$ # datapoints

$$X = \begin{bmatrix} | & | & | & \dots & | \\ x_1 & x_2 & x_3 & \dots & x_n \\ | & | & | & \dots & | \end{bmatrix}_{d \times n}$$

$$X \in \mathbb{R}^{d \times n}$$

$$C = \frac{1}{n} \left(\sum_{i=1}^n x_i x_i^T \right)$$

$XX^T =$
 $\begin{bmatrix} | & \dots & | \\ x_1 & \dots & x_n \\ | & \dots & | \end{bmatrix} \begin{bmatrix} \text{---} x_1 \text{---} \\ \vdots \\ \text{---} x_n \text{---} \end{bmatrix}$

$\Rightarrow$

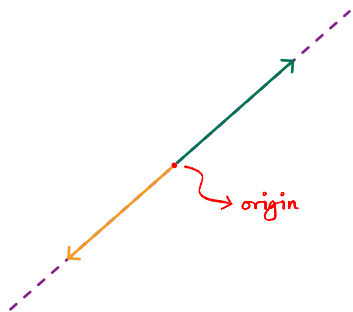
$$C = \frac{1}{n} XX^T$$

Scatter Matrix

$$= \sum_{i=1}^n x_i x_i^T$$

capturing similarities
b/w f_i and f_i

exercise:
[Show this]



$$\text{Square Matrix } A \cdot v = \lambda v$$

Eigenvalue Eigenvector

Let w_k be the eigenvector ^{with length 1} corresponding to the k^{th} largest eigenvalue of $C(\lambda_k)$ _{↑ p-value}

- Consider a matrix as a transformation that takes a vector and moves it to a new position. When this transformation is applied to a space, most vectors will get knocked off their original span and point in entirely new directions.

However, **Eigenvectors** refuse to be knocked off their line. When the transformation hits them, they might get stretched, squished, or reversed, but their underlying direction remains exactly the same.

Eigenvalues are the scale factors by which these eigenvectors are stretched, squished, or reversed.

$$C w_k = \lambda_k w_k \quad [\text{by definition}]$$

$$\left(\frac{1}{n} \sum_{i=1}^n x_i x_i^T \right) w_k = \lambda_k w_k \quad (\text{Associativity})$$

$$w_k = \sum_{i=1}^n \left(\frac{x_i^T w_k}{n \lambda_k} \right) \cdot x_i \quad ; \lambda_k \neq 0$$

constant

- w_k is a LINEAR COMBINATION of data points!
- w_k itself is required to calculate w_k .

$$w_k = \begin{bmatrix} | & & | \\ x_1 & \dots & x_n \\ | & & | \end{bmatrix} \begin{bmatrix} \alpha_{k1} \\ \vdots \\ \alpha_{kn} \end{bmatrix} \\ = \sum_{i=1}^n \alpha_{ki} x_i$$

Data matrix

$$w_k = X \alpha_k$$

for some $\alpha_k \in \mathbb{R}^n$ Co-efficients, different for every round (k)
(vector)How to get α_k ?Some Algebra:

$$w_k = X \alpha_k \quad \forall k$$

$$C w_k = \lambda_k w_k$$

$$\left(\frac{1}{n} X X^T \right) (X \alpha_k) = \lambda_k X \alpha_k$$

$$(XX^T) \alpha_R = n \lambda_R X \alpha_R$$

Pre multiply by X^T

(Associativity)

$$X^T((XX^T) \alpha_R) = X^T(n \lambda_R X \alpha_R)$$

$$(X^T X) (X^T \alpha_R) = n \lambda_R (X^T X) \alpha_R$$

Call $X^T X := K$

$$K \alpha_R = n \lambda_R K \alpha_R$$

$$\begin{aligned} X &\in \mathbb{R}^{d \times n} \\ X^T X &\in \mathbb{R}^{n \times n} \end{aligned}$$

- **Note:** we cannot divide both sides of this equation by K .

or we can find α_R that satisfies

- Here, α_k is not unique. After scaling, it will still satisfy this equation.

$$K \alpha_R = (\lambda_R) \alpha_R \quad \text{--- ①}$$

↳ Eigen equation

we know

$$W_R = X \alpha_R$$

Length² →

$$W_R^T W_R = (X \alpha_R)^T (X \alpha_R) = \alpha_R^T (X^T X) \alpha_R$$

Length constraint on α_k :

$$1 = \alpha_R^T K \alpha_R \quad ; \quad \|W_R\|_2^2 = 1 \quad \text{--- ②}$$

- the α_k must satisfy eq. ① and ②.

- The **rank** of a matrix is the number of linearly independent rows or columns, which equals the number of non-zero rows in its row-echelon form.

LINEAR ALGEBRA FACT : * The non-zero eigenvalues of $\underline{XX^T}$ and $\underline{X^TX}$ are exactly the same!

$\downarrow$
 $\mathbb{R}^{d \times d}$ $\downarrow$
 $\mathbb{R}^{n \times n}$

- * XX^T and X^TX both are **PSD**.
- * $\text{rank}(XX^T) = \text{rank}(X^TX) = \text{rank}(X) = r$.
- * The number of non-zero eigenvalues of these two matrices is equal to the rank r , which is upper bounded by $\min(d, n)$.

$$C = \frac{1}{n} XX^T$$

$$\text{Eigenvectors} = \{w_1, \dots, w_L\}$$

$$\forall k, \|w_k\|^2 = 1$$

$$\text{Eigenvalues} = \{\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_L\}$$

$$XX^T = nC$$

$$\text{Eigenvectors} = \{w_1, \dots, w_L\}$$

$$\text{Eigenvalues} = \{n\lambda_1 \geq n\lambda_2 \geq \dots \geq n\lambda_e\}$$

$$\text{Eigenvectors} = \{ \beta_1, \dots, \beta_e \}$$

$$\|\beta_k\|^2 = 1 \quad \forall k$$

$$\text{Eigenvalues} = \{n\lambda_1 \geq n\lambda_2 \geq \dots \geq n\lambda_e\}$$

$$\frac{X^T X}{K}$$

Kernel Matrix

$$\therefore K\alpha_k = n\lambda_k \alpha_k$$

$$K\beta_k = (n\lambda_k) \beta_k$$

$$\text{IS } \beta_k = \alpha_k?$$

$$\therefore \alpha_k^T K \alpha_k = 1$$

Multiplying both sides by β_k^T : $\beta_k^T K \beta_k = \beta_k^T (n \lambda_k \beta_k) = n \lambda_k \beta_k^T \beta_k \rightarrow \|\beta_k\|_2^2 = 1$

$= n \lambda_k$ (scaling factor)

Set $\alpha_k := \frac{\beta_k}{\sqrt{n \lambda_k}}$ e. vector of $X^T X$

Now (1) $K \alpha_k = (n \lambda_k) \alpha_k$

(2) $\alpha_k^T K \alpha_k = 1$

(Normalisation) $= \frac{\beta_k^T K \beta_k}{n \lambda_k} = \frac{n \lambda_k}{n \lambda_k} = 1$

- **Gram Matrix:** $K = X^T X$; $K \in \mathbb{R}^{n \times n}$
(kernel)

$$K = \begin{bmatrix} -x_1- \\ -x_2- \\ \vdots \\ -x_n- \end{bmatrix} \begin{bmatrix} | & | & \dots & | \\ x_1 & x_2 & \dots & x_n \\ | & | & \dots & | \end{bmatrix}$$

capturing similarities
b/w x_i and x_j $\rightarrow$

$\forall i, j$

$$k_{ij} = x_i^T x_j$$

- Eigen decomposition cannot be avoided.

Input: $D = \{x_1, \dots, x_n\}$ $x_i \in \mathbb{R}^d$ $[d \gg n]$

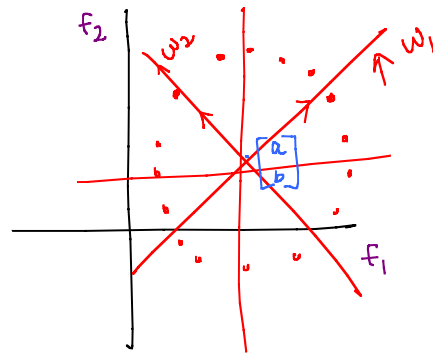
- Step 1: Compute $K = X^T X$ $K \in \mathbb{R}^{n \times n}$
- Step 2: Compute eigendecomposition of K
e.vectors $\{B_1, \dots, B_r\}$ corresponding to e-values $\{\lambda_1, \dots, \lambda_r\}$ $O(n^3)$

- Step 3 $\alpha_R = \frac{B_R}{\sqrt{\lambda_R}}$ $\forall R = 1, \dots, r$

- Step 4: $w_R = X \alpha_R$ $\forall R$.
Co-efficients (vector)

* Co-efficients (vector) = $\frac{\text{Unit eigenvector associated with the } k^{\text{th}} \text{ largest eigenvalue of } K}{\sqrt{k^{\text{th}} \text{ largest eigenvalue of } K}}$

Issue 2 → Non-linear relationships



- After centering the dataset, PCA tries to find the direction of maximum variance. However, since the data points are distributed approximately along a circle, the variance is more or less same in every direction. As a result, all directions are equally important for the first Principal Component.

What would PCA give? $\{w_1, w_2\}$ and both important.

Relation between features

Equation of a circle centered at (a, b) → $(f_1 - a)^2 + (f_2 - b)^2 = r^2$ → radius

$$\Rightarrow \boxed{f_1^2 + a^2 - 2f_1 \cdot a + f_2^2 + b^2 - 2f_2 \cdot b - r^2 = 0} \quad \leftarrow (*)$$

$$\text{or, } (a^2 + b^2 - r^2) + f_1^2 + f_2^2 - 2f_1 a - 2f_2 b = 0$$

- All possible non-linear relationships are captured by ϕ .

Feature Transformation: $\begin{matrix} \text{mapping} \downarrow \\ x \in \mathbb{R}^2 \end{matrix} \xrightarrow{\phi} \begin{matrix} \phi(x) \in \mathbb{R}^6 \end{matrix}$

$\begin{bmatrix} f_1 & f_2 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & f_1^2 & f_2^2 & f_1 f_2 & f_1 & f_2 \end{bmatrix}$

$\mathbb{R}^2$: Original feature space $\mathbb{R}^6$: Transformed feature space

Let $u \in \mathbb{R}^6$ $\begin{bmatrix} a^2 + b^2 - 2c^2 & 1 & 1 & 0 & -2a & -2b \end{bmatrix}$

Each datapoint satisfies $\boxed{\phi(x)^T u = 0} \Rightarrow$ the datapoints lie in a LINEAR subspace of $\mathbb{R}^6$!

$\equiv (*)$

IDEA: Transform features from low dimension $\mathbb{R}^d$ to high dimension $\mathbb{R}^D$

$x \xrightarrow{\phi} \phi(x)$
 $\mathbb{R}^d \quad \quad \mathbb{R}^D$

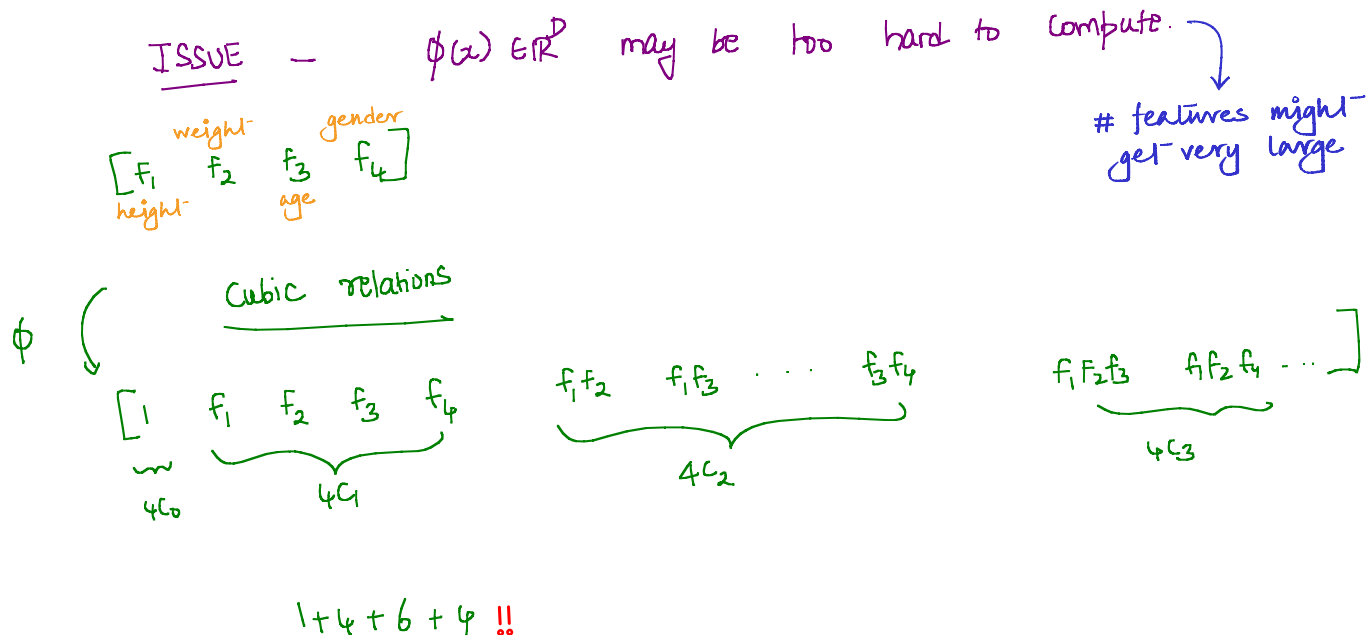
• $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^D$; $d \ll D$

- We already know how to handle the case when $D \gg n$

• $\phi(x) \rightarrow$ Applying the map to every vector in the data matrix

$\overset{K}{\phi(x)^T \phi(x)}$ instead of $\overset{n \times n}{\phi(x) \phi(x)^T}$

- All possible non-linear relationships are captured by ϕ .



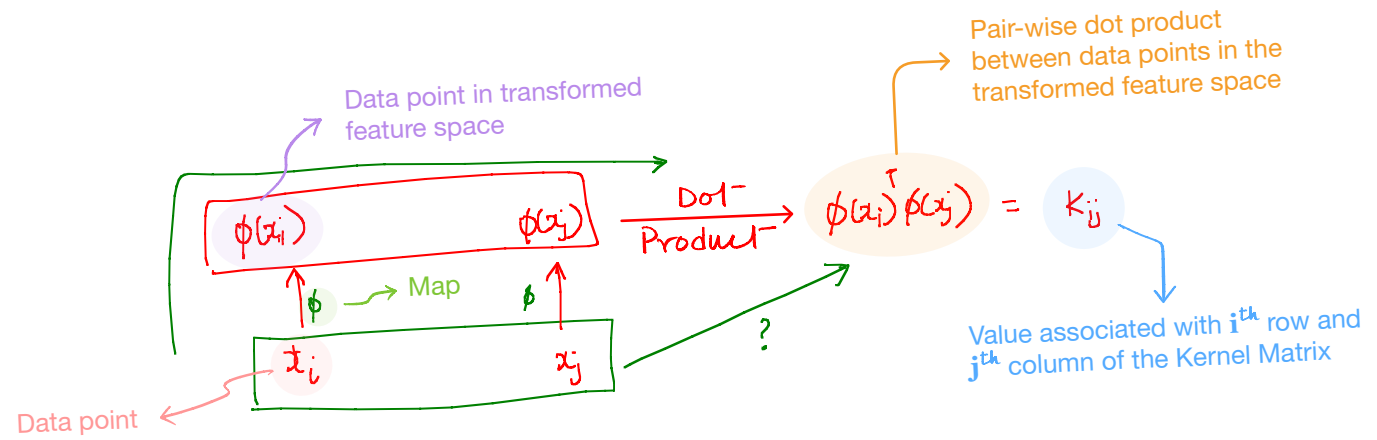
In general, d features, $\leq p$ -th power

$$\sum_{i=0}^p d C_i \approx O(d^p)$$

Dimension of the transformed feature space for a Polynomial Kernel of degree p would grow by roughly an order of d^p

* For Polynomial Kernel, $D = \binom{p+d}{d}$

Degree of the polynomial
 Dimension of the transformed feature space
 Dimension of the original feature space



Example.

$$x = \begin{bmatrix} f_1 & f_2 \end{bmatrix}$$

$$x' = \begin{bmatrix} g_1 & g_2 \end{bmatrix}$$

Consider the function

$$\begin{aligned} (x^T x' + 1)^2 &= \left(\begin{bmatrix} f_1 & f_2 \end{bmatrix} \begin{bmatrix} g_1 \\ g_2 \end{bmatrix} + 1 \right)^2 = \left(f_1 g_1 + f_2 g_2 + 1 \right)^2 \\ &= f_1^2 g_1^2 + f_2^2 g_2^2 + 1 + 2 f_1 g_1 f_2 g_2 + 2 f_1 g_1 + 2 f_2 g_2 \end{aligned}$$

$$\bullet (a+b+c)^2 = a^2 + b^2 + c^2 + 2ab + 2bc + 2ac$$

$$\boxed{(x^T x' + 1)^2}$$

$\swarrow \quad \searrow$
 $[f_1 \ f_2] \quad [g_1 \ g_2]$

$$= \begin{bmatrix} f_1^2 & f_2^2 & 1 & \sqrt{2} f_1 f_2 & \sqrt{2} f_1 & \sqrt{2} f_2 \end{bmatrix} \begin{bmatrix} g_1^2 \\ g_2^2 \\ 1 \\ \sqrt{2} g_1 g_2 \\ \sqrt{2} g_1 \\ \sqrt{2} g_2 \end{bmatrix}$$

$$= \phi(x)^T \phi(x')$$

where

$$\phi(x) = \phi\left(\begin{bmatrix} a \\ b \end{bmatrix}\right) = \begin{bmatrix} a^2 \\ b^2 \\ 1 \\ \sqrt{2}ab \\ \sqrt{2}a \\ \sqrt{2}b \end{bmatrix}$$

INSIGHT:

$(x^T x' + 1)^2$ computes the dot-product in a "transformed space".

$$\begin{aligned} [f_1 \ f_2] &\rightarrow [f_1^2 \ f_2^2 \ 1 \ \sqrt{2} f_1 f_2 \ \sqrt{2} f_1 \ \sqrt{2} f_2] \\ [g_1 \ g_2] &\rightarrow [g_1^2 \ g_2^2 \ 1 \ \sqrt{2} g_1 g_2 \ \sqrt{2} g_1 \ \sqrt{2} g_2] \end{aligned}$$

$$\boxed{\begin{array}{c} x \in \mathbb{R}^2 \\ \downarrow \\ \phi(x) \in \mathbb{R}^6 \end{array}}$$

Kernel Trick:

WE MANAGED TO COMPUTE $\phi(x)^T \phi(x')$ WITHOUT EXPLICITLY
COMPUTING $\phi(x)$

MORE EXAMPLES

Polynomial Kernel: $k(x, x') = (x^T x' + 1)^p$

$x, x' \in \mathbb{R}^d$
for some $p \geq 1$

- Exhibiting an explicit feature map is a sufficient condition to prove a kernel's validity.

→ can be shown to be a "valid" function

i.e., $\exists \phi : \mathbb{R}^d \rightarrow \mathbb{R}^D$ such that
 $k(x, x') = \phi(x)^T \phi(x')$

EXERCISE :

compute the explicit
 ϕ for $p=3$ and
 $p=4$

ONE MORE EXAMPLE

Gaussian Kernel: $k(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right)$ for some $\sigma > 0$

↑
RADIAL BASIS
FUNCTION.

→ Can be shown to be a "valid" map.

→ Interestingly, ϕ in this case maps x to an
"infinite" dimensional space.

[Technicalities aside, can think of this as mapping a point to
a continuous function and dot-products between continuous functions become integrals].
(inner product)

$$\exists \phi: \mathbb{R}^d \rightarrow \mathbb{R}^D \text{ such that,}$$

$$K(x, x') = \phi(x)^T \phi(x') ; x, x' \in \mathbb{R}^d$$

KERNEL FUNCTION

Any function $K: \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ which is a valid map is called a kernel function

$$K(x, x') = (x^T x' + 1)^p \Rightarrow \text{POLYNOMIAL KERNEL}$$

$$K(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right) \Rightarrow \text{RADIAL BASIS / GAUSSIAN KERNEL}$$

constant

Given a function $K: \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$, how can we say it's a valid kernel?

Laplacian Kernel: $K(x, x') = \exp\left(-\frac{\|x - x'\|}{\sigma}\right); \sigma > 0$

constant

METHOD 1: Exhibit a map ϕ explicitly.
 [might be hard sometimes]

METHOD 2: MERCER'S THEOREM (Informal)

$k: \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ is a valid kernel

if and only if

As the dot product is commutative,

$$\langle \phi(x), \phi(x') \rangle = \langle \phi(x'), \phi(x) \rangle$$

i.e., $k(x, x') = k(x', x)$.

(a) k is symmetric

$\left. \begin{matrix} \phi(x)^T \phi(x) \\ \phi(x) \phi(x)^T \end{matrix} \right\}$ Non-zero eigenvalues of these two matrices are the same

All e-values of k are non-negative.

(b) For any dataset $\{x_1, \dots, x_n\}$, the matrix

$K \in \mathbb{R}^{n \times n}$

where

$$K_{ij} = k(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

IS POSITIVE SEMI DEFINITE

also symmetric

$$k(x_i, x_j) = k(x_j, x_i)$$

$$v^T K v \geq 0; \forall v \in \mathbb{R}^n$$

KERNEL PCA

• Input - $\{x_1, \dots, x_n\}$ $x_i \in \mathbb{R}^d$; Kernel $k: \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$.

• Step 1: Compute $K \in \mathbb{R}^{n \times n}$ where
 $k_{ij} = k(x_i, x_j)$



• "Center the Kernel"

Mimicking two steps: going to the higher dimensional feature space and centering the dataset there

• Step 2: Compute $\beta_1, \dots, \beta_l$ eigen vectors of K .
 $n\lambda_1 \geq \dots \geq n\lambda_l$ eigen values

and normalize to get

$$\alpha_u = \frac{\beta_u}{\sqrt{n\lambda_u}}$$

* Given the Kernel Matrix K ,

$$K^c = K - K1_{n \times n} - 1_{n \times n}K + 1_{n \times n}K1_{n \times n}$$

where $1_{n \times n}$ is a $(n \times n)$ matrix with all the elements equal to $1/n$

and K^c is the Centered Kernel Matrix.

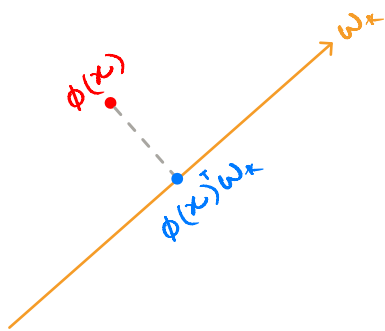
• $\emptyset$ does not make sure that the transformed data points in the higher dimension would also be centered.

X. Step 3:

$w_k = \phi(x) \alpha_k \rightarrow$ Defeats the purpose because it needs $\phi(x)$

• We cannot "reconstruct" the eigenvectors of the co-variance matrix.

• But we can still compute the "compressed" representation.



Projection of $\phi(x)$ onto w after kernel PCA is being applied and ϕ is the mapping that transforms data points into a higher dimensional feature space

$$\phi(x_i)^T w_k$$

$$= \phi(x_i)^T \left(\sum_{j=1}^n \phi(x_j) \alpha_{kj} \right)$$

$$= \sum_{j=1}^n \alpha_{kj} \phi(x_i)^T \phi(x_j) = \sum_{j=1}^n \alpha_{kj} K_{ij}$$

(supervised)
For downstream tasks, this is usually good enough.

- Although explicitly deriving the principal components in the transformed feature space is not feasible, computing and storing the magnitudes of the projections instead of the actual data points is possible. Therefore, reconstructing the proxies in the higher dimensional feature space is not achievable.

$$K = \begin{bmatrix} & \begin{matrix} j=1 & j=2 & \dots & j=n \end{matrix} \\ \begin{matrix} i \\ \vdots \\ \vdots \end{matrix} & \begin{bmatrix} & & & \\ & & & \\ & & & \\ & & & \end{bmatrix} \end{bmatrix}_{n \times n}$$

- Taking the i^{th} row of the kernel matrix and checking similarities of the i^{th} data point with each of the other data points ($j=1, j=2, \dots, j=n$) with respect to this kernel. Then weighing these similarities with the α s and storing them as the most important information corresponding to x_i .

Modified

Step 3 :

Compute

$$\sum_{j=1}^n \alpha_{kj} K_{ij} \quad \forall \quad k$$

- l is the number of rounds.

$$x_i \in \mathbb{R}^d$$

$\rightarrow$

$$\begin{bmatrix} \sum_{j=1}^n \alpha_{1j} K_{1j} & \sum_{j=1}^n \alpha_{2j} K_{2j} & \dots & \sum_{j=1}^n \alpha_{lj} K_{lj} \end{bmatrix}$$

low dimensional representation of the mapped data points

- First we are mapping the original data points into a high dimensional transformed feature space to capture the non-linear relationship, $\mathbb{R}^d \rightarrow \mathbb{R}^D$

And then performing PCA in the transformed feature space to reduce the dimension of the data, $\mathbb{R}^D \rightarrow \mathbb{R}^l$

Therefore, this process might actually increase the original dimension of the dataset to a higher dimensional representations. However, it is necessary to capture the non-linear structure of the data.

DETAILS :

(Centering the kernel)

- Centering the kernel without explicitly computing $\phi(X)$ is possible.

$$\text{Given } K \in \mathbb{R}^{n \times n}$$

$$K_{ij} = K(x_i, x_j) \quad \forall i, j$$

Create a new kernel $K^c \leftarrow \text{centered}$

$$K_{ij}^c = K_{ij} - \theta_i \mathbf{1}_j^T - \mathbf{1}_i \theta_j^T + P \mathbf{1}_i \mathbf{1}_j^T$$

where

$$\theta_i = \frac{1}{n} \sum_{k=1}^n K_{ik}$$

$$P = \frac{1}{n^2} \sum_{i,j} K_{ij}$$

$$\begin{array}{l} \{x_1, \dots, x_n\} \\ \downarrow \\ \{\phi(x_1), \dots, \phi(x_n)\} \\ \downarrow \\ \{\phi(x_1) - \mu, \dots, \phi(x_n) - \mu\} \\ \mu = \frac{1}{n} \sum_i \phi(x_i) \end{array} \quad \left(\begin{array}{l} (\phi(x_i) - \mu)^T (\phi(x_j) - \mu) \\ \boxed{\phi(x_i)^T \phi(x_j)} \\ -\mu^T \phi(x_j) \\ -\mu^T \phi(x_i) \\ + \mu^T \mu \end{array} \right)$$

- Pair-wise dot products between data points imply the possibility of kernelisation.